DATA COMPRESSION

- LOSSLESS
  - Naive
  - Huffman coding
  - LZW ... coding (-> zip)
- LOSSY (mp3, mp4, JPG, PNG, ...)

Alphabet: $\Sigma = \{\overbrace{\sigma_1, \ldots, \sigma_d}^{\text{letters}}\}$ $(0 < d < \infty)$ d: constant.

Text: $T[0..n)$: this is compressed

$T: \Sigma[n]$

or T is a sequential file of Σ.

Naive method: {fixed code length of letters (b)
{compress letter by letter

T = BABRAKADABRA

Latin1 coding: 12*8 = 96 bits ($d = 2^8 = 256$)

Go through the text and identify the letters.

$2^b \geq d > 2^{b-1}$

↳ $b = \lceil \log d \rceil$

$d = 5 \Rightarrow b = 3$

in our example

letters	CODES
B	100
A	000
R	101
K	110
D	111

code table

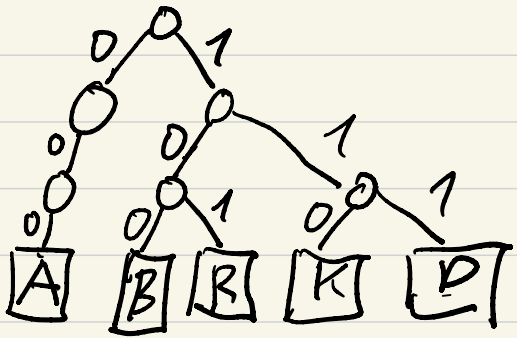
T = BABRAKADABRA

compressed code } 100000100...

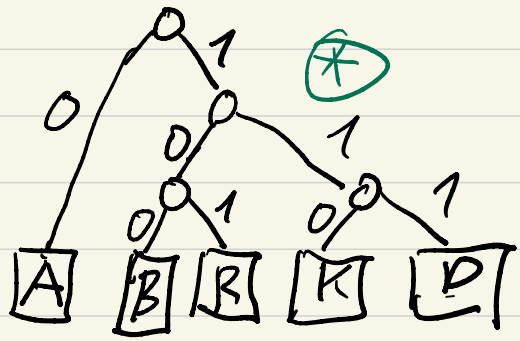
12*3 = 36 bits + code table

compressed file

Code tree



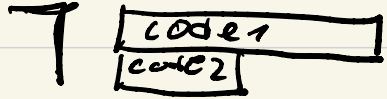
Code tree (reduced)



OPTIMIZATION

Problem: this is intuitive method (not an Alg.)

- still we code letter by letter
- varying code lengths of the letters
- The code is prefix-free because the letters are on the leaves of the code tree.



→ BABRAKADABRA } compression
1000100101010...

Decompression: based on *

1000 | 100 | 1010 | 0...
B A B R A ...

length of compressed file:
5*1+7*3=
26 bits + code tree

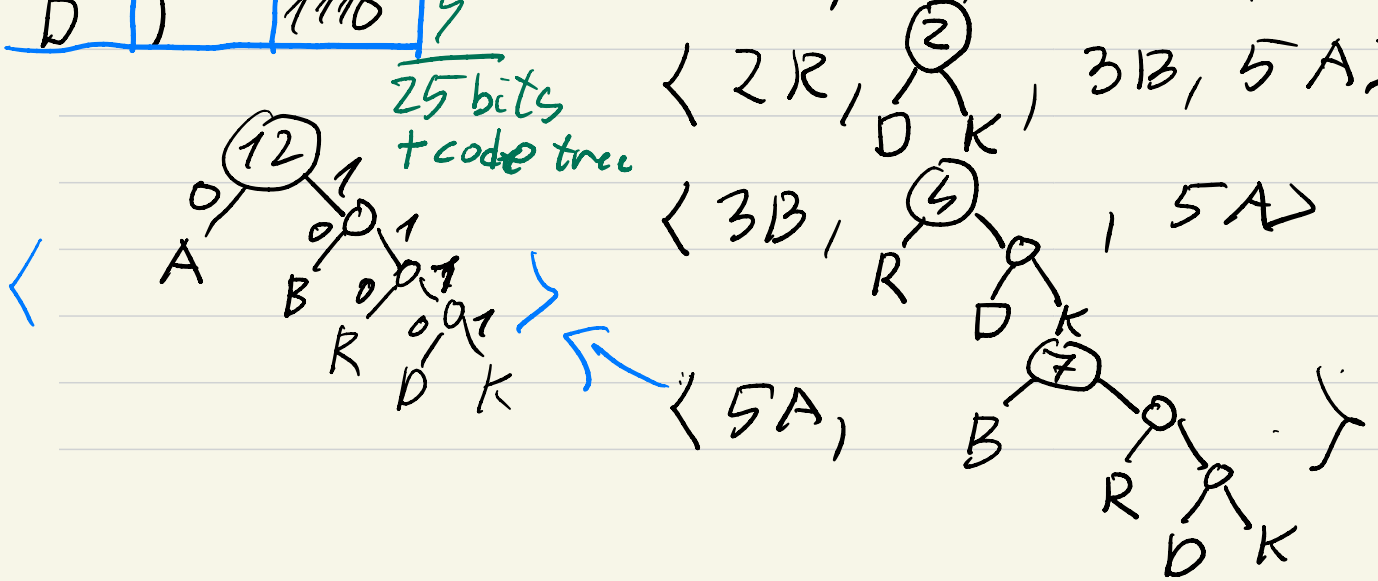
HUFFMAN coding

(letter by letter
varying code length
prefix-free code
characters with higher frequency
have shorter (never longer) code

letter	freq.	code	bits
B	III	10	6
A	~~HH~~	0	5
R	II	110	6
K	I	1111	4
D	I	1110	4

25 bits
+ code tree

PrQ: ⟨1D, 1K, 2R, 3B, 5A⟩



Compression: $T = \underset{10}{B}\ \underset{0}{A}\ \underset{10}{B}\ \underset{110}{R}\ \underset{0}{A}\ K\ A\ D\ A\ B\ R\ A$ ...

(with the code table)

Decompression: $10\ 0\ 10\ 110\ 0\ \ldots$

(with the code tree) $B\ A\ B\ R\ A\ \ldots$

Note: See another example in the notes of the 2nd group.

Theorems:

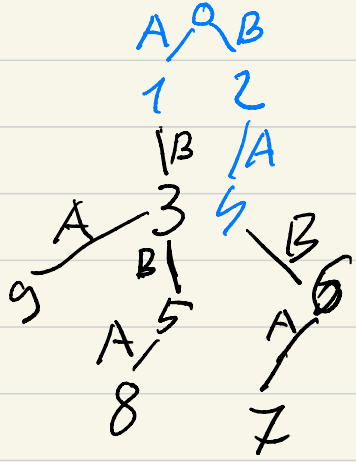
invariability: ① The length of the code does not depend on the way of resolving the nondeterminism of Huffman coding.

optimality: ② The length of a Huffman code is minimal among the character based prefix-free codes of a given text.

LZW compression (Lempel-Ziv-Welch) $\Sigma = \{A, B\}$

D	A	B	AB	BA	ABB	BAB	BABA	ABBA	ABA
	1	2	3	4	5	6	7	8	9

Input	A	B	A	B	B	A	B	A	B	A	B	B	A	B	A	B	A
Output	1		2		3		4		6		5		3		9		



word tree
(prefix tree)

Fixed code length
(in practice: 11–15 bits)
for example: if the code length is 12 bits ⇒
⇒ we have $2^{12} = 4096$ different codes.

LZW
HUFFMAN } { compress / decompress } in $\Theta(n)$ time.

LZW Decompression

LZW is Heuristic alg.

	1	2	3	4	5	6	7	8	9
D	A	B	AB	BA	ABB	BAB	BABA	ABBA	ABA

Input:	1	2	3	4	6	5	3	9
Output:	A	B	AB	BA	BAB	ABB	AB	ABA

Added to D : BA.
Next input string: BA.

AB.
AB.

On average LZW generates 30% shorter code than Huffman

! HUFFMAN reads the input twice (for compression)
• LZW ——"—— once ——"——

↑
In practice LZW compression is faster (zip/unzip are based on LZW)